

DATA ANALYSIS FOR SOCIAL
RESEARCH

Python and Google Colab

M.Sc. Alfonso Rodriguez

September 2026

LESSON 1

From simple values to reusable rules and a portable ENAHO workflow.

What you will be able to do

LEARNING GOALS

Each skill is introduced with a small example, then connected to the ENAHO notebook.

Create variables

Store numbers, text, and logical values with clear names.

Build functions

Package a calculation with parameters and a return value.

Apply functions

Reuse rules with values, lists, and dataframe columns.

Work in Colab

Navigate cells, manage the runtime, and access Drive.

LESSON FLOW

Five sections with a clear break between each one

01	Creating variables	Values, types, and decisions
02	Building functions	def, parameters, return
03	Applying functions	Values, lists, dataframes
04	Colab notebook layout	Text, code, output
05	Colab essentials	Runtime, packages, Drive
LAB	ENAH0 practice	Three portable CSV files

Three files support one social-data question

ENAH0 2023 is the practice context.

How are education and hourly earnings descriptively related among salaried workers in Peru?

Person key

ano, conglome, vivienda, hogar, codperso

Module 200

119,747 people. Household membership, sex, age, and stratum.

Module 300

108,354 people age 3+. Educational level, year, and grade.

Module 500

86,654 people age 14+. Work, hours, pay, and survey weight.

01

Creating variables

Names let us reuse values. Start with one person before asking Python to work across thousands of ENAHO records.

A variable stores a value under a name

Assignment uses a single equals sign.

```
edad = 32
anios_educacion = 11
ingreso_mensual = 1800.0
trabajo_semana_pasada = True
```

edad is the variable name.

32 is the stored value.

The name sits to the left of =.

Running the cell creates the variable in memory.

Four basic types appear everywhere

PYTHON TYPES

Python infers the type from the value. `type(variable)` reveals it.

int

Whole numbers such as `edad = 32`.

float

Numbers with decimals such as `ingreso = 1800.0`.

str

Text inside quotes such as `sexo = 'Mujer'`.

bool

Logical values True or False.

Expressions create new values

Operators combine variables without changing the original data.

```
experiencia = edad - años_educacion - 6
```

```
ingreso_hora = ingreso_mensual / (48 * 4.33)
```

```
es_adulto = edad >= 18
```

Arithmetic returns a number.

Comparisons return True or False.

Parentheses make the intended order visible.

Use descriptive names for every result.

Collections keep related values together

A list preserves order. A dictionary connects keys with values.

```
variables_clave = [  
    'edad',  
    'anios_educacion',  
    'ingreso_mensual'  
]  
  
persona = {  
    'sexo': 'Mujer',  
    'edad': 32,  
    'anios_educacion': 11  
}
```

Use a list when position or sequence matters.

Use a dictionary when every value needs a label.

`persona['edad']` returns 32.

A dataframe row behaves like a labeled record.

Conditions turn values into decisions

Indentation tells Python which lines belong to each branch.

```
if horas_semana > 0:  
    ingreso_hora = ingreso_mensual / (horas_semana  
    * 4.33)  
else:  
    ingreso_hora = None
```

if checks a condition.

else handles the remaining cases.

Four spaces are the usual indentation.

None represents the absence of a valid result.

Practice with one fictional person

PREDICT BEFORE YOU RUN

```
edad = 27
anios_educacion = 16
ingreso_mensual = 2400.0
horas_semana = 40
```

1. Predict

What type does each value have?

2. Calculate

Create experiencia and ingreso_hora.

3. Explain

Which assumption changes the hourly result?

02

Building functions

A function gives a name to a rule, identifies its inputs, and returns a result that other code can use.

A function turns a rule into reusable code

READ THE DEFINITION

```
def calcular_ingreso_hora(ingreso, horas):  
    if horas <= 0:  
        return None  
    return ingreso / (horas * 4.33)
```

def

Names the function and begins its definition.

parameters

Receive the values used by the calculation.

return

Sends the result back to the calling code.

Parameters receive names

Arguments provide the actual values when the function is called.

```
def calcular_experiencia(edad, educacion):  
    experiencia = edad - educacion - 6  
    return max(experiencia, 0)  
  
resultado = calcular_experiencia(32, 11)
```

Inside the definition

edad and educacion are parameters.

Inside the call

32 and 11 are arguments.

The names make the calculation general.

return and print serve different purposes

A reusable function returns a value. Printing is optional feedback.

```
def duplicar(valor):  
    return valor * 2  
  
resultado = duplicar(5)  
print(resultado)
```

return gives 10 back to resultado.

print displays 10 below the cell.

A printed value is not automatically stored.

Prefer return for analytical functions.

A function can include decisions

Conditions keep invalid cases away from the calculation.

```
def calcular_ingreso_hora(ingreso, horas):  
    if ingreso <= 0 or horas <= 0:  
        return None  
    return ingreso / (horas * 4.33)
```

The guard clause appears first.

or means either condition is enough.

Valid inputs reach the final return.

Invalid inputs produce None.

Protect the meaning of the result

A calculation can run and still be conceptually wrong.

```
calcular_ingreso_hora(1800, 48)
```

```
# 8.66
```

```
calcular_ingreso_hora(1800, 0)
```

```
# None
```

```
calcular_ingreso_hora(-50, 40)
```

```
# None
```

Check units before coding.

Define which inputs are valid.

Document the 4.33 convention.

Use a clear missing-result rule.

A useful function has four qualities

FUNCTION CHECKLIST

Review the rule before applying it to a dataframe.

One task

The name describes a single calculation.

Clear inputs

Parameters reveal the required information.

Visible checks

Invalid or missing cases are handled explicitly.

Returned result

Other code can store, compare, or summarize it.

Practice: complete the function

Write the missing condition and calculation before running the tests.

```
def calcular_experiencia(edad, educacion):  
    # Complete the rule  
    ...  
  
assert calcular_experiencia(32, 11) == 15  
assert calcular_experiencia(18, 16) == 0
```

Expected rule

$edad - educacion - 6$

Lower bound

The result cannot be negative.

Useful tool

$\max(\text{resultado}, 0)$

03

Applying functions

Defining a function stores the rule. Calling or applying it performs the rule with specific data.

Call a function with parentheses

The returned value can be stored, rounded, compared, or displayed.

```
ingreso_hora = calcular_ingreso_hora(1800, 48)

print(round(ingreso_hora, 2))

# 8.66
```

The function name selects the rule.

Arguments fill the parameters.

Parentheses perform the call.

The returned value becomes ingreso_hora.

Reuse a function across several values

A loop or comprehension repeats the same transparent rule.

```
edades = [24, 32, 45]
educacion = [11, 16, 6]

experiencias = [
    calcular_experiencia(e, a)
    for e, a in zip(edades, educacion)
]
```

zip pairs values in the same position.

The comprehension calls the function once per pair.

The result is a new list.

The original lists remain available.

apply can send each row to a function

Use axis=1 when the function needs several columns from one row.

```
muestra['experiencia_potencial'] = muestra.apply(  
    lambda fila: calcular_experiencia(  
        fila['p208a'],  
        fila['anios_educacion_aprox']  
    ),  
    axis=1  
)
```

muestra.apply repeats a rule.

lambda receives one row as fila.

Column labels provide the arguments.

The returned values form a new column.

Vectorized expressions are often simpler

Pandas can calculate directly across entire columns.

```
muestra['mujer'] = (  
    muestra['p207'] == 2  
)<code>.astype(int)  
  
muestra['ingreso_hora_nominal'] = (  
    muestra['ingreso_mensual_nominal']  
    / (muestra['p513t'] * 4.33)  
)
```

Column operations avoid an explicit Python loop.

Comparisons create Boolean series.

`astype(int)` converts True and False to 1 and 0.

Keep parentheses around multi-line expressions.

Derived variables preserve the research logic

ENAH0 EXAMPLES

```
muestra['experiencia_potencial'] = (  
    muestra['p208a']  
    - muestra['anios_educacion_aprox']  
    - 6  
) .clip(lower=0)
```

Sex indicator

mujer equals 1 when p207 equals 2.

Area

estrato 1 to 5 is labeled Urbana, otherwise Rural.

Hourly income

Monthly nominal income divided by weekly hours and 4.33.

Check a new column before trusting it

VALIDATION HABITS

A function or expression can produce values without producing the intended values.

head()

Inspect several rows and source columns together.

dtype

Confirm numbers are numeric and labels are text.

isna()

Count missing results after the transformation.

describe()

Review range, center, and suspicious extremes.

Practice: apply the rule to records

Compare a direct call, a list comprehension, and a dataframe expression.

```
registros = [  
    {'ingreso': 1800, 'horas': 48},  
    {'ingreso': 2400, 'horas': 40},  
    {'ingreso': 900, 'horas': 0}  
]  
  
# Create ingreso_hora for each record
```

Which record returns None?

Which approach is easiest to read?

What assumptions should accompany the result?

How would these records become dataframe rows?

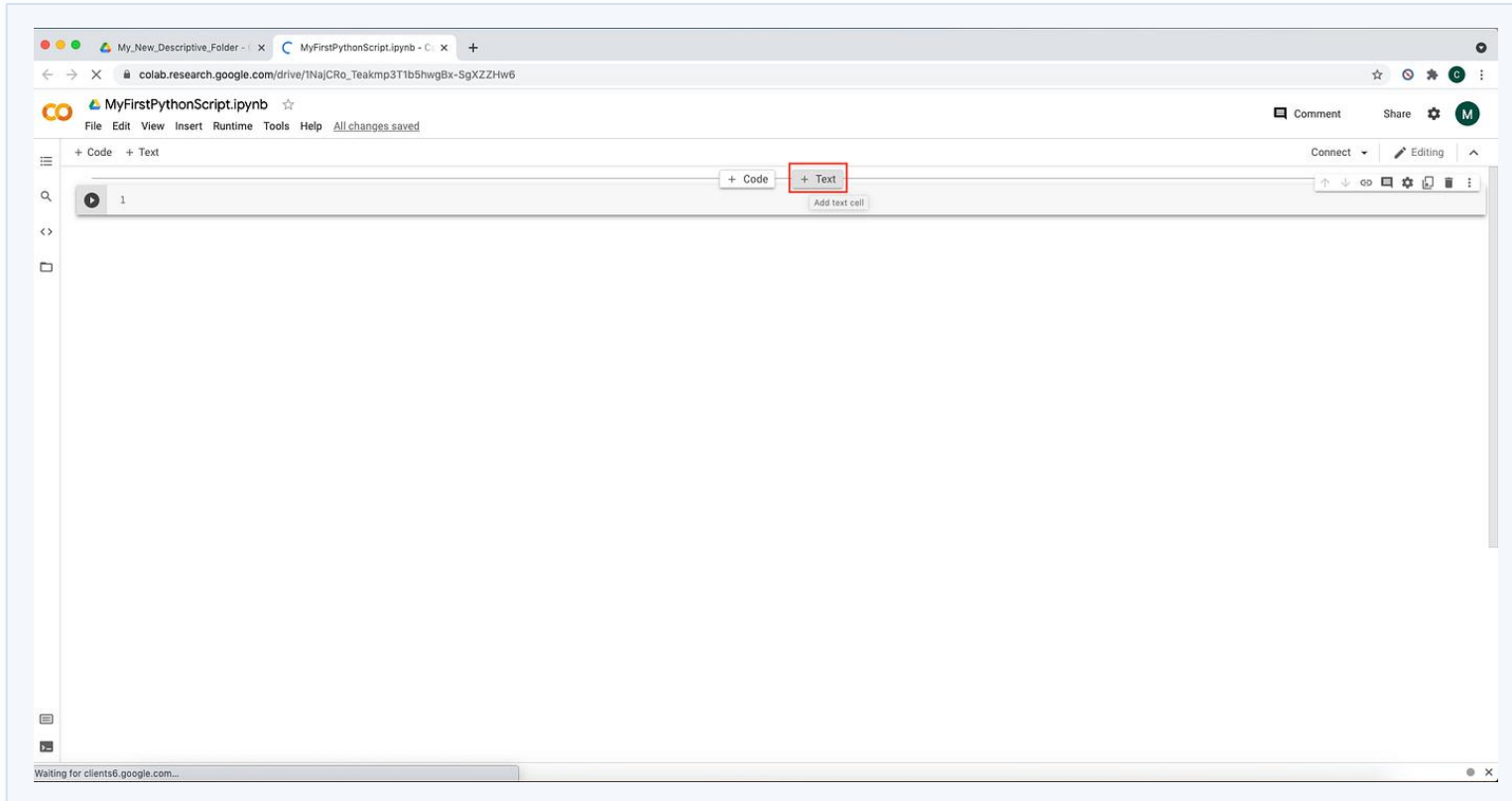
04

Google Colab layout

A notebook combines explanation, executable Python, visible output, and files in one ordered document.

A notebook alternates text, code, and output

Read the explanation. Run the code. Interpret the output.



Text cells explain the reasoning

Code cells run Python

Output appears below the code

The runtime keeps variables in memory

Three elements create the notebook flow

READ, RUN, INTERPRET

A notebook is an ordered sequence. Cells can be inserted, moved, deleted, or rerun without rewriting the whole document.

Text cell

Uses Markdown for headings, instructions, links, and interpretation.

Code cell

Contains executable Python and a run button on the left.

Output

Appears below the code and records the result of that execution.

Execution order controls the current state

The runtime remembers what has run, even when cells appear in another order.

```
edad = 32
edad
# 32

# After restarting the runtime
edad
# NameError: name 'edad' is not defined
```

The bracket number shows execution history.

Variables exist only after their cell runs.

Editing a cell does not update later outputs automatically.

Restart and run all when state becomes confusing.

The sidebars help you navigate

COLAB LAYOUT

Open panels with the icons on the left edge of the notebook.

Table of contents

Jump between Markdown headings in a long lesson.

Find and replace

Search code, variable names, and instructions.

Files

Inspect /content and files available to the runtime.

Variables

Review objects currently stored in memory when available.

Practice: orient yourself before coding

OPEN THE STUDENT NOTEBOOK

```
notebooks/  
01_sesion_1_estudiantes_enaho.ipynb
```

```
data/  
enaho_2023_200_clase.csv  
enaho_2023_300_clase.csv  
enaho_2023_500_clase.csv
```

1. Locate

Find the first Python section in the table of contents.

2. Identify

Point to one text cell, one code cell, and one output area.

3. Verify

Open Files after mounting Drive and find the data folder.

05

Colab essentials

The runtime performs the work. Drive stores the lesson. A few controls make the workflow predictable.

Four controls solve most runtime tasks

CORE CONTROLS

Use the toolbar and Runtime menu deliberately.

Run

Execute the selected cell with Shift + Enter.

Interrupt

Stop a cell that is taking too long.

Restart

Clear variables and begin with a fresh runtime.

Run all

Execute the notebook from top to bottom in order.

Import packages before using them

Install only when the runtime does not already provide the package.

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Only if a package is missing
!pip install package_name
```

import loads a package into the runtime.

as gives a short conventional alias.

Commands beginning with ! run in the system shell.

A restarted runtime may require installation again.

Mount Google Drive

The mount makes MyDrive available to the temporary Colab runtime.

```
from google.colab import drive
from pathlib import Path

drive.mount('/content/drive')

MY_DRIVE = Path('/content/drive/MyDrive')
```

Run the mount once per fresh runtime.

Approve access to the intended Google account.

MY_DRIVE becomes the stable starting folder.

Drive files persist after the runtime stops.

Define one lesson path

Every later file path starts from the same folder.

```
CARPETA_LECCION = (  
    MY_DRIVE  
    / 'Courses'  
    / '2026'  
    / 'Taught courses'  
    / 'Bootcamp data for social analysis'  
    / '01_python'  
)  
  
RUTA_DATOS = CARPETA_LECCION / 'data'
```

Path joins folders safely.

Change only CARPETA_LECCION if the folder moves.

RUTA_DATOS points to the included CSV files.

Keep notebooks and data inside 01_python.

Confirm all three CSV files before reading

Fail early with a useful message when the folder is incomplete.

```
NOMBRES_DATOS = [  
    'enaho_2023_200_clase.csv',  
    'enaho_2023_300_clase.csv',  
    'enaho_2023_500_clase.csv'  
]  
  
archivos = [RUTA_DATOS / n for n in NOMBRES_DATOS]  
faltantes = [p.name for p in archivos if not  
p.exists()]  
assert not faltantes, f'Faltan: {faltantes}'
```

The list states the required inputs.

`exists()` checks the mounted file system.

The comprehension collects missing names.

`assert` stops the workflow before `read_csv` fails later.

Save the work in the right place

PERSISTENCE AND COLLABORATION

The runtime is temporary. The notebook in Drive is the lasting document. Confirm that edits finish saving before closing the tab.

Save

Colab autosaves changes to the notebook stored in Drive.

Share

Use Drive permissions when another person needs access.

Download

Export an .ipynb copy when you need a portable backup.

Common problems have simple checks

TROUBLESHOOTING

Read the complete error message and identify the first failing assumption.

NameError

Run the cell that creates the missing variable.

FileNotFoundError

Check the mount and CARPETA_LECCION path.

ModuleNotFoundError

Import or install the missing package.

Stale output

Restart the runtime, then run all cells in order.

From three ENAHO modules to one analysis table

01	Locate files	Confirm the portable folder
02	Read modules	Normalize column names
03	Inspect units	Person and household
04	Validate keys	Check duplicates
05	Join sources	Use explicit cardinality
06	Create variables	Describe hourly income

Hands-on challenge

USE THE INCLUDED ENAHO FILES

Open the student notebook and complete the first Python practice. Then run the location cell and confirm that all three CSV paths exist.

Evidence 1

Show four variables with the expected types.

Evidence 2

Call `calcular_ingreso_hora` with valid and invalid hours.

Evidence 3

Display the three ENAHO filenames from `RUTA_DATOS`.

The workflow is now reusable

RECAP

Keep these four ideas visible as the notebook becomes more complex.

Variables store

Names make values and assumptions reusable.

Functions organize

Parameters, checks, and return define a rule.

Application scales

Calls, comprehensions, and columns repeat the rule.

Colab coordinates

Cells, runtime state, and Drive keep the work connected.